

Sistema Distribuído de Monitoramento e Controle de Tráfego Urbano Inteligente

Bruno Braga, Caio Gomes, Kaio Henrique Lucio e Santos, Lucas Araujo
Nalberth Vieira, Pedro Henrique Bellone, Vitor Alexandre

¹ Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Abstract. *This paper presents a distributed urban traffic monitoring and control system built in three layers: an edge layer with React/Tailwind sensors, a Node.js Bridge API, and a gRPC-based processing core. The system implements Lamport clocks, the Bully election algorithm, and cascade synchronization to address scalability and resilience challenges in smart cities.*

Resumo. *Este artigo apresenta um sistema distribuído de monitoramento de tráfego urbano em três camadas: sensores web em React/Tailwind, uma Bridge API em Node.js e um núcleo via gRPC. O sistema implementa relógios de Lamport, o algoritmo Bully e sincronização em cascata para tratar desafios de escalabilidade e resiliência em cidades inteligentes.*

1. Introdução

Sistemas centralizados de controle de semáforos apresentam limitações críticas: um único ponto de falha pode paralisar regiões inteiras e a alta latência dificulta reações em tempo real [Tanenbaum and Van Steen 2007]. A computação distribuída elimina esses problemas ao distribuir a inteligência por múltiplos nós autônomos [Coulouris et al. 2011]. Este trabalho propõe um **sistema distribuído de monitoramento e controle de tráfego urbano**, onde cada nó gerencia uma zona da cidade e coopera com os demais para detectar congestionamentos e ajustar semáforos automaticamente.

As principais contribuições deste trabalho são:

- Arquitetura distribuída em três camadas com comunicação via gRPC;
- Implementação de relógios lógicos de Lamport para ordenação causal de eventos;
- Implementação do algoritmo Bully para eleição de líder e tolerância a falhas;
- Mecanismo de sincronização em cascata entre nós vizinhos.

O restante deste artigo está organizado da seguinte forma: a Seção 2 descreve o problema abordado; a Seção 3 detalha a arquitetura do sistema; a Seção 4 apresenta a implementação dos requisitos técnicos; a Seção 5 discute os desafios encontrados; e a Seção 7 apresenta as conclusões e trabalhos futuros.

2. Descrição do Problema

O problema central é o **controle adaptativo e resiliente de semáforos urbanos** em múltiplas zonas. O sistema deve ser capaz de:

1. Receber dados em tempo real de sensores de contagem de veículos;
2. Detectar automaticamente congestionamentos com base em limiares predefinidos;

3. Ajustar os tempos de semáforo das zonas afetadas e vizinhas;
4. Continuar operando mesmo que um ou mais nós falhem;
5. Preservar a sequência temporal dos eventos mesmo com atrasos de rede.

A natureza geográfica do problema torna-o um candidato natural à abordagem distribuída: cada zona é gerenciada por um nó independente que se comunica com os vizinhos apenas quando necessário, reduzindo o tráfego de rede e aumentando a autonomia local.

3. Arquitetura do Sistema

3.1. Camada de Borda (Sensores/Semáforos)

A camada de borda é simulada por uma **interface web em React com Tailwind CSS**, representando o painel de um semáforo inteligente. Os dispositivos atuam como **nós sensores**, permitindo entrada de dados em tempo real por operadores de campo ou integração com sensores físicos em implantações futuras.

3.2. Camada de Integração (Bridge API)

A Bridge API é um serviço em **Node.js** que traduz protocolos entre a camada de borda e o núcleo distribuído. Ela expõe uma **API HTTP/REST**, recebe os dados dos sensores e os injeta no cluster, encaminhando cada leitura ao nó responsável pela zona correspondente. Essa separação mantém o isolamento entre as camadas.

3.3. Camada de Processamento Distribuído (Core)

O núcleo é composto por **múltiplos nós independentes** que se comunicam via **gRPC**. Cada nó gerencia uma zona geográfica e executa as seguintes funções:

- Receber e processar leituras de tráfego da Bridge API;
- Gerar alertas quando o fluxo ultrapassa limiares predefinidos;
- Manter e incrementar seu relógio lógico de Lamport;
- Participar do processo de eleição de líder;
- Notificar nós vizinhos sobre congestionamentos detectados.

3.3.1. Troca de Mensagens entre Nós

A comunicação entre os nós ocorre exclusivamente via gRPC. As principais mensagens são:

1. **TrafficAlert**: detecta congestionamento; contém zona, contagem de veículos, timestamp de Lamport e severidade;
2. **ElectionMessage**: utilizada na eleição de líder; contém o identificador e timestamp do nó candidato;
3. **CoordinatorMessage**: anuncia o novo líder eleito a todos os nós;
4. **NeighborSync**: sincronização em cascata para ajuste imediato dos tempos de semáforo nos nós vizinhos.

4. Implementação dos Requisitos Técnicos

4.1. Comunicação via gRPC

A comunicação entre nós utiliza **gRPC** com **Protocol Buffers (Protobuf)** para definição dos contratos de serviço, garantindo alta performance, tipagem estrita e interoperabilidade. A escolha se justifica pelo menor overhead de serialização, suporte a streaming bidirecional e clareza dos contratos [gRPC Authors 2024].

4.2. Relógios Lógicos de Lamport

Para garantir a **ordenação causal** dos eventos, o sistema implementa os **relógios lógicos de Lamport** [Lamport 1978]:

- Antes de enviar uma mensagem: $C_i \leftarrow C_i + 1$;
- Ao receber uma mensagem com timestamp t : $C_i \leftarrow \max(C_i, t) + 1$.

Isso permite reconstruir a sequência causal correta dos congestionamentos mesmo com atrasos de rede variáveis.

4.3. Algoritmo de Eleição de Líder (Bully)

O **Algoritmo Bully** [Garcia-Molina 1982] garante a resiliência do sistema elegendo um Coordenador responsável por consolidar alertas e gerar relatórios globais. A eleição ocorre na inicialização ou quando um nó não recebe o *heartbeat* do Coordenador dentro do timeout configurado. O nó iniciante envia **ELECTION** aos nós de maior ID; se nenhum responder, declara-se Coordenador e notifica os demais com **COORDINATOR**.

4.4. Sincronização em Cascata

Ao detectar tráfego pesado, um nó notifica automaticamente seus vizinhos via `NeighborSync`. Os nós vizinhos ajustam preventivamente seus tempos de semáforo, antecipando o fluxo proveniente da área congestionada.

5. Desafios e Problemas Encontrados

Configuração do gRPC: O alinhamento de versões das bibliotecas entre os módulos do projeto exigiu tempo considerável durante a configuração inicial.

Falha vs. lentidão: Distinguir um nó falho de um sobrecarregado foi um desafio de design; o valor de timeout do heartbeat precisou de ajuste empírico para evitar eleições desnecessárias.

Consistência do relógio de Lamport: Situações de alta frequência de mensagens simultâneas exigiram atenção especial ao controle de concorrência dentro de cada nó.

Integração entre camadas: A Bridge API foi projetada como *stateless* para não introduzir um novo ponto único de falha no sistema.

6. Contribuição dos Alunos

A condução do trabalho foi dividida da seguinte forma entre os membros do grupo:

- **Bruno Braga:** Arquitetura geral do sistema e contratos Protobuf;
- **Caio Gomes:** Implementação dos nós do núcleo distribuído em Node.js;

- **Kaio Henrique Lucio:** Implementação dos relógios lógicos de Lamport;
- **Lucas Araujo:** Implementação do algoritmo de eleição Bully;
- **Nalberth Vieira:** Testes de integração e validação do sistema;
- **Pedro Henrique Bellone:** Configuração do ambiente e documentação;
- **Vitor Alexandre:** Desenvolvimento da Bridge API e interface web em React.

7. Conclusão

Este trabalho demonstrou como conceitos de computação distribuída — comunicação via RPC, ordenação causal com relógios lógicos e eleição de líder — podem ser aplicados a um problema urbano real. A arquitetura proposta detecta congestionamentos, propaga ajustes em cascata e se recupera de falhas de forma autônoma. Como trabalhos futuros, pretende-se:

- Implementar replicação de estado entre os nós para maior disponibilidade;
- Avaliar o desempenho frente à execução centralizada;
- Integrar sensores físicos reais à camada de borda.

Referências

- Coulouris, G., Dollimore, J., Kindberg, T., and Blair, G. (2011). *Distributed Systems: Concepts and Design*. Addison-Wesley, 5 edition.
- Garcia-Molina, H. (1982). Elections in a distributed computing system. In *IEEE Transactions on Computers*, volume 31, pages 48–59.
- gRPC Authors (2024). gRPC: A high-performance, open source universal RPC framework. <https://grpc.io>. Acessado em: maio de 2025.
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565.
- Tanenbaum, A. S. and Van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2 edition.